

02 - Arquitectura del Backend (NestJS)

Documento Técnico

Arquitectura del Backend del Sistema POS Web para Boticas

1. Objetivo

Definir la arquitectura del Backend, su organización interna, las responsabilidades de cada capa y las reglas de desarrollo.

El Backend será el núcleo del sistema.

Será responsable de:

- Reglas de negocio
- Seguridad
- Autenticación
- Autorización
- Validaciones
- Auditoría
- Sincronización
- Eventos
- Persistencia
- Integraciones externas

El Frontend nunca implementará lógica crítica.

2. Responsabilidades

El Backend será la única fuente de verdad.

Será responsable de:

- Validar datos
 - Validar permisos
 - Gestionar sesiones
 - Gestionar dispositivos
 - Administrar el carrito compartido
 - Mantener sincronizados todos los clientes
 - Emitir eventos
 - Registrar auditoría
 - Ejecutar transacciones
-

3. Arquitectura

```
graph TD; Cliente --> REST_Socket[REST / Socket]; REST_Socket --> Authentication; Authentication --> Middlewares; Middlewares --> Guards_JWT[Guards JWT]; Guards_JWT --> Interceptors; Interceptors --> Controllers; Controllers --> Services; Services --> Repositories; Repositories --> Prisma; Prisma --> PostgreSQL;
```

Cliente

|

REST / Socket

|

Authentication

|

Middlewares

|

Guards JWT

|

Interceptors

|

Controllers

|

Services

|

Repositories

|

Prisma

|

PostgreSQL

4. Organización del Proyecto

```
src/  
  
├─ auth/  
├─ users/  
├─ companies/  
├─ branches/  
├─ terminals/  
├─ sessions/  
├─ devices/  
├─ dashboard/  
├─ products/  
├─ inventory/  
├─ customers/  
├─ suppliers/  
├─ laboratories/  
├─ categories/  
├─ purchases/  
├─ sales/  
├─ payments/  
├─ cash/  
├─ reports/  
├─ notifications/  
├─ socket/  
├─ sunat/  
├─ audit/  
├─ config/  
├─ common/  
├─ database/  
└─ main.ts
```

5. Organización de un Módulo

Todos los módulos tendrán exactamente la misma estructura.

Ejemplo

```
products/  
  
├─ controllers/  
├─ services/  
├─ repositories/  
├─ dto/  
├─ entities/  
├─ events/  
├─ gateways/  
├─ validators/  
└─ interfaces/
```

```
|— constants/  
|— products.module.ts
```

Esto facilita el mantenimiento.

6. Controllers

Responsabilidades

- Recibir peticiones
- Validar DTO
- Invocar servicios
- Retornar respuestas

Nunca deberán:

- Consultar la base de datos
- Contener lógica de negocio
- Ejecutar cálculos

Ejemplo

Cliente

↓

Controller

↓

Service

7. Services

Aquí vive toda la lógica del negocio.

Ejemplos

- Crear producto
- Registrar venta
- Calcular stock
- Abrir caja
- Cerrar caja
- Calcular descuentos
- Validar reglas

Todo deberá pasar por Services.

8. Repositories

Responsables únicamente del acceso a datos.

Ejemplo

```
ProductoService
↓
ProductoRepository
↓
Prisma
↓
PostgreSQL
```

Nunca contendrán reglas de negocio.

9. DTO

Todos los datos entrarán mediante DTO.

Ejemplo

```
CreateProductDto
UpdateProductDto
CreateSaleDto
LoginDto
CreateCustomerDto
```

Nunca utilizar directamente entidades.

10. Entities

Representan el modelo del dominio.

No representan tablas.

Representan objetos del negocio.

Ejemplo

Producto

Cliente

Venta

Caja

Empresa

11. Interfaces

Toda comunicación interna utilizará interfaces.

Ejemplo

ProductRepository

SessionManager

InventoryService

NotificationService

Nunca depender directamente de implementaciones.

12. Validaciones

Se utilizarán tres niveles.

DTO

Formato.

Ejemplo

Email

Longitud

Número

Texto requerido

Service

Reglas del negocio.

Ejemplo

No vender sin stock.

No modificar una venta pagada.

No eliminar un usuario administrador.

Base de Datos

Restricciones.

Ejemplo

UNIQUE

FOREIGN KEY

CHECK

NOT NULL

13. Autenticación

JWT

Flujo

Usuario

↓

Login

↓

JWT

↓

Cliente

↓

Cada petición

↓

Bearer Token

↓

Backend

↓

Validación

14. Autorización

Cada usuario tendrá:

- Rol
- Permisos
- Empresa
- Sucursal
- Caja

Nunca se confiará en el Frontend.

15. Gestión de Empresas

Todas las consultas deberán ejecutarse dentro del contexto de una empresa.

Ejemplo

Empresa

↓

Productos

↓

Ventas

↓

Clientes

Nunca mezclar datos entre empresas.

16. Gestión de Sucursales

Cada empresa podrá tener múltiples sucursales.

Empresa

|

├─ Lima

├─ Ica

├─ Pisco

└─ Chincha

Cada operación conocerá la sucursal.

17. Gestión de Cajas

Cada sucursal tendrá varias cajas.

Sucursal

|

├─ Caja 01

└─ Caja 02

└─ Caja 03

Cada caja tendrá una sesión independiente.

18. Gestión de Sesiones

Cada usuario solamente podrá tener una sesión activa.

La sesión contendrá

Usuario

Empresa

Sucursal

Caja

Estado

Dispositivos

Carrito

Fecha Inicio

19. Gestión de Dispositivos

Cada sesión podrá registrar dispositivos.

Ejemplo

PC Principal

Celular

Tablet

Cada dispositivo tendrá

- ID
- Nombre

- Tipo
- Última actividad
- Estado

20. Gestión del Carrito

El carrito NO pertenecerá al navegador.

Pertenecerá al Backend.

Sesión



Carrito



Productos



Totales

Todos los dispositivos consultarán el mismo carrito.

21. Socket Gateway

Responsabilidades

- Conectar clientes
- Desconectar clientes
- Administrar Rooms
- Emitir eventos
- Sincronizar dispositivos

Nunca acceder directamente a la base de datos.

22. Eventos

Cada operación importante emitirá eventos.

Ejemplo

```
producto.creado  
  
producto.actualizado  
  
producto.eliminado  
  
venta.creada  
  
venta.pagada  
  
venta.anulada  
  
stock.actualizado  
  
cliente.creado  
  
precio.actualizado  
  
sesion.iniciada  
  
sesion.finalizada
```

23. Auditoría

Toda operación importante quedará registrada.

Ejemplo

```
Usuario  
  
↓  
  
Modificar Precio  
  
↓  
  
Guardar Auditoría  
  
↓  
  
Actualizar Producto
```

Registrar

- Usuario
- Fecha
- Hora

- IP
- Dispositivo
- Acción
- Datos anteriores
- Datos nuevos

24. Manejo de Errores

Nunca devolver errores internos.

Ejemplo incorrecto

```
Prisma Error...
```

Ejemplo correcto

```
{  
  "success": false,  
  "message": "No existe stock suficiente."  
}
```

25. Transacciones

Toda operación crítica utilizará transacciones.

Ejemplo

Registrar Venta

```
Crear Venta
```

```
↓
```

```
Descontar Stock
```

```
↓
```

```
Registrar Pago
```

```
↓
```

```
Registrar Auditoría
```

↓

Emitir Evento

Si una falla

↓

Rollback

26. Integraciones

El Backend será responsable de integrar

- SUNAT
- WhatsApp
- Email
- Impresoras
- APIs externas
- Balanzas
- Lectores
- Facturación electrónica

Nunca el Frontend.

27. Seguridad

Implementar

- JWT
- Refresh Token
- Helmet
- Rate Limit
- CORS
- Hash BCrypt
- HTTPS
- Validaciones
- Sanitización
- Auditoría

28. Configuración

Toda configuración vivirá en

```
.env  
  
ConfigModule  
  
ConfigurationService
```

Nunca utilizar valores hardcodeados.

29. Principios

Todo el Backend deberá cumplir

- Responsabilidad Única
 - Inversión de Dependencias
 - Open/Closed
 - Bajo Acoplamiento
 - Alta Cohesión
 - Arquitectura Modular
 - Event Driven
 - Clean Architecture (adaptada)
 - Domain Driven Design (parcial)
-

30. Resultado Esperado

El Backend será capaz de:

- Atender cientos de usuarios concurrentes.
- Sincronizar dispositivos en tiempo real.
- Mantener un único carrito compartido por sesión.
- Gestionar empresas, sucursales y cajas.
- Proteger la información mediante autenticación y autorización.
- Escalar sin modificar la arquitectura principal.
- Integrarse con nuevos módulos y servicios externos sin afectar el núcleo del sistema.